

# Building a Modern Data Warehouse from Scratch

Integrating CRM and ERP Source Systems using the Medallion  
Architecture

**Presented by: Amira Bhar**  
Data Engineering Project

Academic Year: 2025/2026

# Contents

|          |                                                           |           |
|----------|-----------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction and Context</b>                           | <b>3</b>  |
| 1.1      | What is a Data Warehouse?                                 | 3         |
| 1.2      | Motivation: Manual Reporting vs. Automated Data Warehouse | 3         |
| 1.2.1    | Scenario 1 – Without a Data Warehouse                     | 3         |
| 1.2.2    | Scenario 2 – With a Data Warehouse                        | 4         |
| 1.2.3    | Roles Involved                                            | 4         |
| 1.3      | The Restaurant Analogy                                    | 4         |
| <b>2</b> | <b>ETL: The Core Component of Data Warehousing</b>        | <b>6</b>  |
| 2.1      | Overview of ETL                                           | 6         |
| 2.2      | Step-by-Step Breakdown                                    | 6         |
| 2.2.1    | Extract                                                   | 6         |
| 2.2.2    | Transform (Heavy Processing Step)                         | 6         |
| 2.2.3    | Load                                                      | 6         |
| 2.3      | ETL in Real Projects: Multiple Layers                     | 7         |
| 2.4      | Extraction (E) – Getting Data from Source                 | 7         |
| 2.4.1    | Extraction Directions                                     | 7         |
| 2.4.2    | Extraction Types                                          | 7         |
| 2.4.3    | Extraction Techniques                                     | 7         |
| 2.5      | Transformation (T) – Shaping the Data                     | 7         |
| 2.6      | Load (L) – Writing to Target                              | 7         |
| 2.6.1    | Processing Types                                          | 7         |
| 2.6.2    | Load Methods                                              | 7         |
| 2.7      | Slowly Changing Dimensions (SCD) – Handling History       | 8         |
| <b>3</b> | <b>Data Management Approaches and Architecture Design</b> | <b>10</b> |
| 3.1      | The Four Major Data Management Approaches                 | 10        |
| 3.1.1    | Quick Decision Guide                                      | 10        |
| 3.2      | Building the DWH: The Four Approaches                     | 11        |
| 3.2.1    | The Inmon Approach (“Slow & Complete”)                    | 11        |
| 3.2.2    | The Kimball Approach (“Fast & Focused”)                   | 11        |
| 3.2.3    | The Data Vault Approach (“Standards & Rules”)             | 11        |
| 3.2.4    | The Medallion Architecture (Simple & Modern)              | 11        |
| 3.3      | Separation of Concerns (SOC)                              | 12        |
| <b>4</b> | <b>Naming Conventions</b>                                 | <b>13</b> |
| 4.1      | General Principles                                        | 13        |
| 4.2      | Table Naming Conventions                                  | 13        |
| 4.2.1    | Bronze Rules                                              | 13        |
| 4.2.2    | Silver Rules                                              | 13        |
| 4.2.3    | Gold Rules                                                | 13        |

|          |                                                                          |           |
|----------|--------------------------------------------------------------------------|-----------|
| 4.2.4    | Glossary of Category Patterns . . . . .                                  | 14        |
| 4.3      | Column Naming Conventions . . . . .                                      | 14        |
| 4.3.1    | Surrogate Keys . . . . .                                                 | 14        |
| 4.3.2    | Technical Columns . . . . .                                              | 14        |
| 4.4      | Stored Procedures . . . . .                                              | 14        |
| <b>5</b> | <b>Project Implementation</b>                                            | <b>16</b> |
| 5.1      | Database and Schema Setup . . . . .                                      | 16        |
| 5.2      | Building the Bronze Layer . . . . .                                      | 16        |
| 5.2.1    | Source System Analysis . . . . .                                         | 16        |
| 5.2.2    | Data Ingestion . . . . .                                                 | 16        |
| 5.3      | Building the Silver Layer . . . . .                                      | 16        |
| 5.3.1    | Data Analysis and Integration . . . . .                                  | 16        |
| 5.3.2    | Metadata Columns . . . . .                                               | 17        |
| 5.3.3    | Data Quality Checks and Cleansing . . . . .                              | 17        |
| 5.3.4    | Assembling the Silver Layer Procedure . . . . .                          | 22        |
| 5.4      | Building the Gold Layer . . . . .                                        | 22        |
| 5.4.1    | Bronze/Silver vs. Gold . . . . .                                         | 22        |
| 5.4.2    | Data Modeling . . . . .                                                  | 23        |
| 5.4.3    | Documentation Types . . . . .                                            | 23        |
| 5.4.4    | Facts and Dimensions . . . . .                                           | 23        |
| 5.4.5    | Creating the Customer Dimension ( <code>dim_customers</code> ) . . . . . | 23        |
| 5.4.6    | Creating the Product Dimension ( <code>dim_products</code> ) . . . . .   | 25        |
| 5.4.7    | Creating the Sales Fact Table ( <code>fact_sales</code> ) . . . . .      | 25        |
| 5.4.8    | Building the Star Schema Model . . . . .                                 | 26        |
| 5.4.9    | Creating the Data Catalog . . . . .                                      | 26        |
| 5.4.10   | Finalizing the Data Flow Diagram . . . . .                               | 27        |
| <b>6</b> | <b>Conclusion</b>                                                        | <b>28</b> |

# Chapter 1

## Introduction and Context

### 1.1 What is a Data Warehouse?

According to Bill Inmon's classical definition, a **Data Warehouse (DWH)** is a system that is:

- **Subject-oriented:** focused on key business areas such as sales, customers, and finance.
- **Integrated:** it combines data coming from multiple heterogeneous source systems.
- **Time-variant:** it stores historical data over time, not just the current snapshot.
- **Nonvolatile:** once data is loaded, it is not deleted or modified.

In simpler terms, a data warehouse is a central storage system that collects and organizes data from multiple sources, transforming it into a clean and historical repository ready for analysis, so that businesses can make better decisions.

### 1.2 Motivation: Manual Reporting vs. Automated Data Warehouse

#### 1.2.1 Scenario 1 – Without a Data Warehouse

In organizations without a data warehouse, the reporting process is manual and chaotic:

- Multiple analysts manually extract data from different systems.
- Each analyst spends days or weeks transforming raw data.
- Reports are shared through inconsistent tools (Excel, PowerPoint, Power BI).

This approach leads to several problems:

- Reports take weeks or months to produce.
- Different consumers see inconsistent versions of the data (e.g., one report is 40 days old, another 10 days old, another 5 days old).
- The manual process is slow, stressful, and error-prone.
- Errors propagate into reports and lead to bad business decisions.
- The process cannot scale to big data volumes.
- Building integrated, cross-source reports manually is practically impossible.

### 1.2.2 Scenario 2 – With a Data Warehouse

A data warehouse solves these issues through an automated **ETL (Extract, Transform, Load)** pipeline, which:

- Extracts data from the source systems.
- Applies the necessary transformations.
- Loads the result into the data warehouse, which becomes the *single point of truth* for analysis and reporting.

The benefits of this approach are summarized below:

- Fully automated process, with no manual steps.
- Reduced human errors.
- Fast turnaround (hours or minutes instead of weeks or months).
- Full integration of all data sources in one place.
- Access to historical data.
- Consistency: all reports reflect the same data freshness (e.g., all data is 1 day old).
- Modern cloud data warehouses can easily handle big data volumes.

### 1.2.3 Roles Involved

Two main roles are involved in a data warehouse project:

| Role          | Responsibility                                                                                         |
|---------------|--------------------------------------------------------------------------------------------------------|
| Data Engineer | Accesses the source systems, scripts the ETL pipeline, and builds the data warehouse database.         |
| Data Analyst  | Consumes the data warehouse, builds data models and reports, and communicates results to stakeholders. |

Table 1.1: Roles involved in a data warehouse project

## 1.3 The Restaurant Analogy

A useful analogy to understand the role of a data warehouse is to compare it to a **kitchen in a restaurant**:

| <b>Restaurant</b>                                                                            | <b>Data Warehouse</b>                                                            |
|----------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------|
| Suppliers bring fresh ingredients (vegetables, spices, meat).                                | Multiple source systems provide raw data.                                        |
| Ingredients are cleaned, chopped, organized, and stored in the right place (fridge/freezer). | The ETL process cleans, transforms, and organizes the data.                      |
| When an order comes in, prepared ingredients are grabbed to create a dish and serve it.      | When a report or analysis is needed, it is served directly from ready-made data. |

Table 1.2: Restaurant analogy for a data warehouse

## Chapter 2

# ETL: The Core Component of Data Warehousing

The **ETL (Extract, Transform, Load)** process is the core element of a data warehouse. In practice, roughly 90% of a data warehouse project's time is spent building and refining the ETL component.

### 2.1 Overview of ETL

| Letter | Step      | Description                                                                          |
|--------|-----------|--------------------------------------------------------------------------------------|
| E      | Extract   | Identify and pull data from the source system (no changes, 1:1 copy).                |
| T      | Transform | Manipulate, reshape, and clean the data.                                             |
| L      | Load      | Insert the transformed data into the target (e.g., database tables, data warehouse). |

Table 2.1: The three steps of ETL

### 2.2 Step-by-Step Breakdown

#### 2.2.1 Extract

- **Goal:** identify which data needs to be pulled from the source.
- **Typical approach:** incremental loads, i.e., loading a subset of data rather than everything.
- **Key rule:** do not change anything during extraction – keep the data as-is, as a 1:1 copy from the source.

#### 2.2.2 Transform (Heavy Processing Step)

- **Goal:** reshape the data into the format required for analysis and reporting.
- **Activities include:** data cleansing, data integration, formatting, and data normalization.

#### 2.2.3 Load

- **Goal:** insert the prepared data into its final destination.
- **Action:** move the transformed data into the target (e.g., the data warehouse tables).

## 2.3 ETL in Real Projects: Multiple Layers

In real architectures (data lake, data warehouse, etc.), data flows through multiple layers. Moving data between layers does not always require a full ETL pipeline – depending on the data architecture design, only a subset of the ETL components may be necessary. In other words, the ETL process used depends entirely on the design of the data architecture.

## 2.4 Extraction (E) – Getting Data from Source

### 2.4.1 Extraction Directions

| Method | Direction                                                    |
|--------|--------------------------------------------------------------|
| Pull   | The ETL tool requests/retrieves data from the source system. |
| Push   | The source system sends data to the data warehouse.          |

Table 2.2: Extraction directions

### 2.4.2 Extraction Types

| Type                   | Description                                                        |
|------------------------|--------------------------------------------------------------------|
| Full Extraction        | Loads all records from all tables every time (e.g., every day).    |
| Incremental Extraction | Identifies and loads only new or changed data since the last load. |

Table 2.3: Extraction types

### 2.4.3 Extraction Techniques

- **Manual:** a person manually accesses and extracts data.
- **Database Query:** connect to the database and use SQL/queries to extract data.
- **File-based:** pass files (CSV, JSON, etc.) to the data warehouse.
- **API:** connect to an API and write code to extract data.
- **Event-based Streaming:** extract data from streaming sources (e.g., Kafka).
- **Change Data Capture (CDC):** similar to streaming, captures changes in real time.
- **Web Scraping:** code is run to extract information from websites.

## 2.5 Transformation (T) – Shaping the Data

## 2.6 Load (L) – Writing to Target

### 2.6.1 Processing Types

### 2.6.2 Load Methods

**Full Load Methods** (loading everything from scratch):



| Transformation Type    | Description                                                                                                                                    |
|------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| Data Enrichment        | Add new values to existing datasets (e.g., adding demographic data).                                                                           |
| Data Integration       | Combine multiple sources into one unified data model.                                                                                          |
| Derive New Columns     | Create new columns based on existing ones (e.g., calculating age from birth date).                                                             |
| Data Normalization     | Map codes to friendly, human-readable values (e.g., “1” → “Active”).                                                                           |
| Business Rules / Logic | Apply company-specific criteria to build new columns or transform data.                                                                        |
| Data Aggregation       | Change the granularity of the data (e.g., daily sales → monthly sales).                                                                        |
| Data Cleansing         | Removing duplicates, filtering data, handling missing or invalid values, removing unwanted spaces, casting data types, and detecting outliers. |

Table 2.4: Transformation types

| Processing Type   | Description                                                                                                                  |
|-------------------|------------------------------------------------------------------------------------------------------------------------------|
| Batch Processing  | Loads data in one large batch on a fixed schedule (e.g., once or twice per day); reports refresh at that time.               |
| Stream Processing | Processes each change from the source as soon as it happens, enabling a real-time data warehouse (challenging to implement). |

Table 2.5: Load processing types

- **Truncate & Insert:** empty the table, then insert all new data.
- **Upsert (Update + Insert):** update existing records and insert new ones.
- **Drop, Create & Insert:** drop the entire table, recreate it, and then insert the data (similar to truncate, but also removes the table structure).

**Incremental Load Methods** (loading only changes):

- **Upsert:** update or insert statements, but applied only to changed records.
- **Insert Only (Append):** always add new rows without updating existing ones (useful for logs).
- **Merge:** similar to upsert, but also includes deletion (Update + Insert + Delete).

## 2.7 Slowly Changing Dimensions (SCD) – Handling History

| SCD Type   | Strategy                                                                                                | History Preserved? |
|------------|---------------------------------------------------------------------------------------------------------|--------------------|
| SCD Type 0 | No changes allowed; data is fixed.                                                                      | No (frozen)        |
| SCD Type 1 | Override/overwrite old values with new information (upsert).                                            | No (history lost)  |
| SCD Type 2 | Insert a new record for each change; the old record is marked as inactive and the new record as active. | Yes (full history) |

Table 2.6: Slowly Changing Dimensions (SCD) strategies

## Chapter 3

# Data Management Approaches and Architecture Design

### 3.1 The Four Major Data Management Approaches

| Approach                             | Best For                                                  | Data Types                                                       | Key Characteristics                                                          | Risk / Note                                         |
|--------------------------------------|-----------------------------------------------------------|------------------------------------------------------------------|------------------------------------------------------------------------------|-----------------------------------------------------|
| Data Warehouse                       | Solid foundations for reporting & BI                      | Only structured data                                             | Traditional, highly organized, centralized                                   | Very stable, only for structured data               |
| Data Lake                            | Advanced analytics & ML                                   | Structured, semi-structured, unstructured (logs, images, videos) | Very flexible but less organized                                             | Can become “Data Swamp” too unorganized             |
| Data Lakehouse (Modern, recommended) | Both BI reporting and advanced analytics / ML             | Mixed types (structured + unstructured)                          | Hybrid – flexibility of a lake + organization of a warehouse                 | Currently most modern approach                      |
| Data Mesh                            | Organizations where a central system becomes a bottleneck | Any                                                              | Decentralized – multiple departments build and share their own data products | No single centralized system; each domain owns data |

Table 3.1: The four major data management approaches

#### 3.1.1 Quick Decision Guide

The choice of architecture depends on the main need of the organization:

- Only structured data and reporting/Bi → **Data Warehouse**.
- Mixed data types (images, logs, video) with ML/analytics needs → **Data Lake**.
- Need for both organization and flexibility → **Data Lakehouse** (recommended).
- Central system becomes a bottleneck → **Data Mesh** (decentralized).

## 3.2 Building the DWH: The Four Approaches

### 3.2.1 The Inmon Approach (“Slow & Complete”)

| Layer                | What it does                                                                   |
|----------------------|--------------------------------------------------------------------------------|
| Staging              | Raw data lands as-is.                                                          |
| Enterprise DWH (3NF) | Data is cleaned, integrated, and organized.                                    |
| Data Marts           | Small, focused subsets (e.g., just sales, just customers), ready for BI tools. |

Table 3.2: The Inmon approach

**Pro:** very organized, single source of truth. **Con:** takes a long time to build.

### 3.2.2 The Kimball Approach (“Fast & Focused”)

This approach skips the central enterprise warehouse and goes straight to data marts.

| Layer      | What it does                                                       |
|------------|--------------------------------------------------------------------|
| Staging    | Raw data lands.                                                    |
| Data Marts | Built immediately for reporting (no central warehouse in between). |

Table 3.3: The Kimball approach

**Pro:** much faster than Inmon. **Con:** can become chaotic over time.

### 3.2.3 The Data Vault Approach (“Standards & Rules”)

This approach splits the middle layer into two parts.

| Layer          | What it does                                |
|----------------|---------------------------------------------|
| Staging        | Raw data lands.                             |
| Raw Vault      | Original data, no changes.                  |
| Business Vault | Business rules and transformations applied. |
| Data Marts     | Ready for reporting.                        |

Table 3.4: The Data Vault approach

**Pro:** very strict standards, great for large and complex projects. **Con:** more complex to build.

### 3.2.4 The Medallion Architecture (Simple & Modern)

This is the architecture chosen for this project. It organizes data into three layers: Bronze, Silver, and Gold.

**Advantages of the Medallion Architecture:**

- Very easy to understand.
- Great traceability – one can always go back to the Bronze layer.
- Flexible – the Gold layer can serve many purposes (BI, ML, AI).
- Modern and widely adopted.

| Layer  | Nickname             | What it does                                                           | Analogy                             |
|--------|----------------------|------------------------------------------------------------------------|-------------------------------------|
| Bronze | Raw layer            | Raw data as-is (like staging).                                         | Dirty ingredients from the farm.    |
| Silver | Clean layer          | Data cleansing, but no business rules yet.                             | Washed and chopped ingredients.     |
| Gold   | Business-ready layer | Business rules applied; ready for reporting, ML, AI, or data products. | The final dish served to customers. |

Table 3.5: The Medallion Architecture

### High Level Architecture

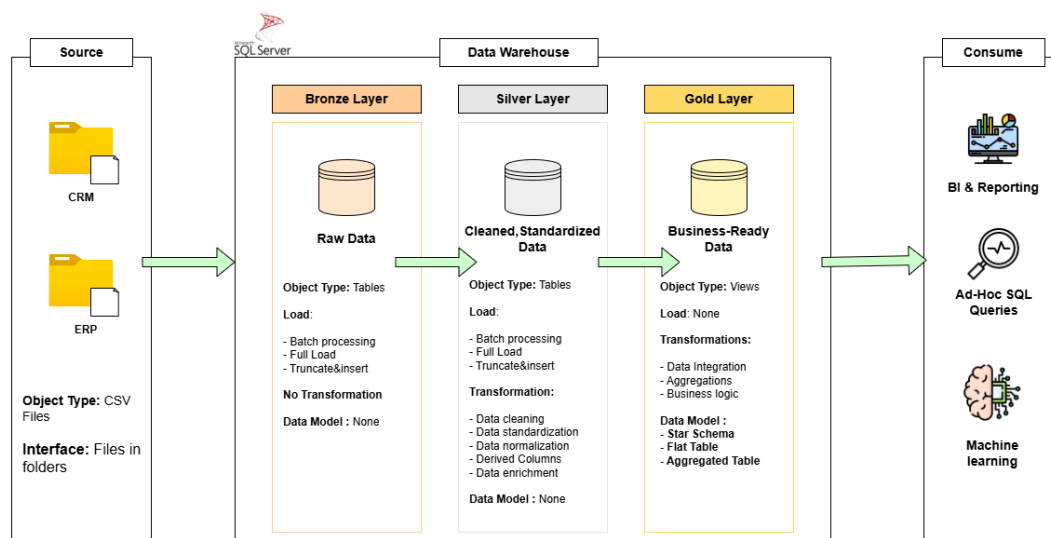


Figure 3.1: Medallion Architecture: from raw data (Bronze) to cleansed data (Silver) and business-ready data (Gold).

### 3.3 Separation of Concerns (SOC)

A key architectural principle applied throughout the project is the **Separation of Concerns (SOC)**: each layer (Bronze, Silver, Gold) has its own independent tasks, and responsibilities are not mixed between layers.

Two important definitions used throughout this project:

- **Load method:** refers to *how* data is written to or updated in database tables when moving it from one place to another (e.g., truncate & insert, upsert, merge).
- **Data modeling:** refers to *how* the structure of the tables is designed and how tables relate to each other.

# Chapter 4

## Naming Conventions

To ensure consistency, traceability, and maintainability, a strict naming convention was defined for all schemas, tables, views, columns, and other objects in the data warehouse.

### 4.1 General Principles

- **Naming convention:** use `snake_case`, with lowercase letters and underscores (`_`) to separate words.
- **Language:** use English for all object names.
- **Reserved words:** avoid using SQL reserved words as object names.

### 4.2 Table Naming Conventions

#### 4.2.1 Bronze Rules

- All table names must start with the source system name, and must match the original table names without renaming.
- Pattern: `<sourcesystem>_<entity>`
  - `<sourcesystem>`: name of the source system (e.g., `crm`, `erp`).
  - `<entity>`: exact table name from the source system.
- Example: `crm_customer_info` → customer information from the CRM system.

#### 4.2.2 Silver Rules

- Same convention as Bronze: table names must start with the source system name and match the original table names.
- Pattern: `<sourcesystem>_<entity>`
- Example: `crm_customer_info` → customer information from the CRM system.

#### 4.2.3 Gold Rules

- All table names must use meaningful, business-aligned names, starting with a category prefix.
- Pattern: `<category>_<entity>`

- **<category>**: describes the role of the table, such as **dim** (dimension) or **fact** (fact table).
- **<entity>**: descriptive name of the table, aligned with the business domain (e.g., **customers**, **products**, **sales**).
- Examples:
  - **dim\_customers** → dimension table for customer data.
  - **fact\_sales** → fact table containing sales transactions.

#### 4.2.4 Glossary of Category Patterns

| Pattern | Meaning         | Example(s)                                |
|---------|-----------------|-------------------------------------------|
| dim_    | Dimension table | dim_customer, dim_product                 |
| fact_   | Fact table      | fact_sales                                |
| report_ | Report table    | report_customers,<br>report_sales_monthly |

Table 4.1: Glossary of category patterns for Gold-layer tables

### 4.3 Column Naming Conventions

#### 4.3.1 Surrogate Keys

- All primary keys in dimension tables must use the suffix **\_key**.
- Pattern: **<table\_name>\_key**
  - **<table\_name>**: name of the table or entity the key belongs to.
  - **\_key**: suffix indicating that this column is a surrogate key.
- Example: **customer\_key** → surrogate key in the **dim\_customers** table.

#### 4.3.2 Technical Columns

- All technical columns must start with the prefix **dwh\_**, followed by a descriptive name indicating the column's purpose.
- Pattern: **dwh\_<column\_name>**
  - **dwh**: prefix exclusively reserved for system-generated metadata.
  - **<column\_name>**: descriptive name indicating the column's purpose.
- Example: **dwh\_load\_date** → system-generated column storing the date when the record was loaded.

### 4.4 Stored Procedures

- All stored procedures used for loading data follow the pattern **load\_<layer>**.
  - **<layer>**: represents the layer being loaded, such as **bronze**, **silver**, or **gold**.
- Examples:

- `load_bronze` → stored procedure for loading data into the Bronze layer.
- `load_silver` → stored procedure for loading data into the Silver layer.



# Chapter 5

## Project Implementation

This chapter describes the practical implementation of the data warehouse, built from two source systems (**CRM** and **ERP**), each containing three tables, following the Medallion Architecture (Bronze, Silver, Gold).

### 5.1 Database and Schema Setup

The first step consisted of creating the database and the three schemas: **bronze**, **silver**, and **gold**. A **schema** is the blueprint or structure of a database: it defines how data is organized, named, and related to each other.

### 5.2 Building the Bronze Layer

#### 5.2.1 Source System Analysis

The first step was to analyze the source systems (CRM and ERP) in order to understand the structure, content, and relationships of their tables before building the ingestion pipeline.

#### 5.2.2 Data Ingestion

The Bronze layer consists of six tables, mirroring the six source tables from CRM and ERP. Two strategies were considered for loading data from the sources into the data warehouse:

- **Bulk Insert:** very fast, because it loads all data in a single operation.
- **Insert:** slow, because it loads data row by row.

A **data flow (data lineage)** diagram was also produced to document how data moves from the source systems into the Bronze layer.

### 5.3 Building the Silver Layer

The Silver layer is where raw Bronze data is cleaned, standardized, and enriched, using a full load (Truncate & Insert), without creating any new data models. This layer prepares the data for the Gold layer, where modeling happens.

#### 5.3.1 Data Analysis and Integration

Before coding the cleansing logic, the Silver layer tables were analyzed and the relationships between CRM and ERP tables were identified, in order to plan the integration logic.

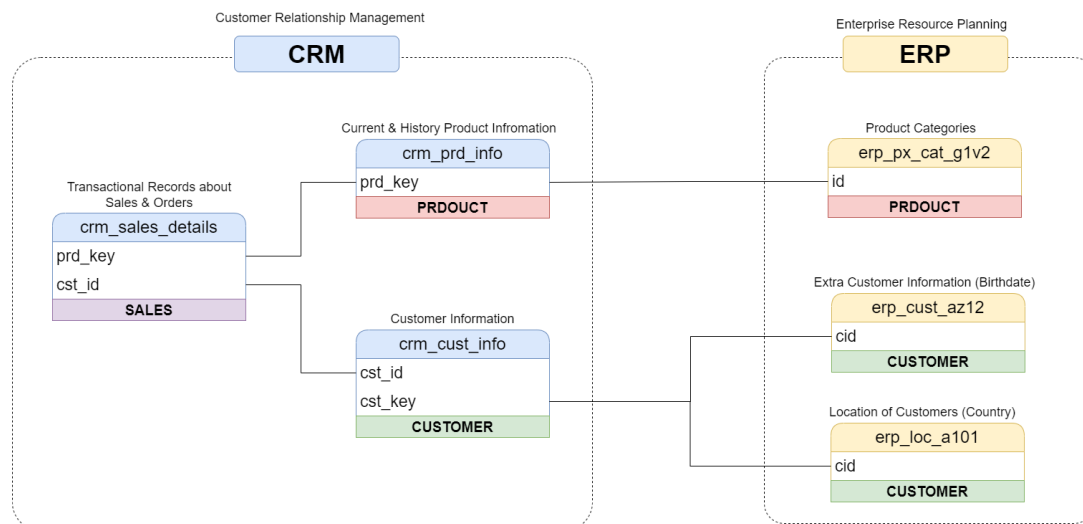


Figure 5.1: Data Analysis and Integration.

### 5.3.2 Metadata Columns

Metadata columns are additional fields added by data engineers (not present in the source systems). They are used to debug and trace corrupt or bad data and are particularly useful for detecting data gaps, especially with incremental loads. Examples include:

- **Create Date:** load time.
- **Update Date:** modification time.
- **Source System:** origin of the data.
- **File Location:** lineage information.

In this project, the Silver-layer tables are the same six tables as in Bronze: the Bronze DDL scripts were reused, with **bronze** replaced by **silver**, and a metadata column was added to each table:

```
1 dwh_create_date DATETIME2 DEFAULT GETDATE()
```

- **DATETIME2:** a data type more precise than **DATETIME** (precision up to 100 nanoseconds).
- **DEFAULT GETDATE():** if no value is provided, the current date and time is used automatically.

### 5.3.3 Data Quality Checks and Cleansing

This subsection details the data quality checks and cleansing operations performed on each table of the Bronze layer before loading the data into the Silver layer.

#### Table: **crm\_cust\_info**

**1. Removing Duplicates** For each customer (**cst\_id**), only the most recent record (by **cst\_create\_date**) should be kept. To identify it, the **ROW\_NUMBER()** window function is used.

After generalizing the logic to all customers (dropping any filter on a specific **cst\_id**), we obtain only the rows where the ID is not null and unique, keeping the row with the newest **create\_date**.

**2. Removing Unwanted Spaces (Data Trimming)** A check is performed to detect unwanted leading or trailing spaces in text columns:

```
1 -- Check for unwanted Spaces
2 -- Expectation: No Results
3 SELECT cst_firstname
4 FROM bronze.crm_cust_info
5 WHERE cst_firstname != TRIM(cst_firstname)
```

The solution is to apply TRIM() to the relevant columns during the load into Silver.

**3. Data Normalization and Standardization** The distinct values of the gender column are first inspected:

```
1 -- Data Standardization & Consistency
2 SELECT DISTINCT cst_gndr
3 FROM bronze.crm_cust_info
```

The goal is to make the values meaningful and user-friendly, by replacing 'M' with 'Male' and 'F' with 'Female'. The same standardization logic is applied to the cst\_material\_status column ('S' → 'Single', 'M' → 'Married').

**4. Date Type Correction** For the create\_date column, it is ensured that the data type is DATE and not VARCHAR.

**5. Loading the Clean Table into the Silver Layer** The final cleansed data is loaded into silver.crm\_cust\_info using the following query, which combines all the cleansing rules described above:

```
1 INSERT INTO silver.crm_cust_info(
2     cst_id,
3     cst_key,
4     cst_firstname,
5     cst_lastname,
6     cst_material_status,
7     cst_gndr,
8     cst_create_date)
9 SELECT
10     cst_id,
11     cst_key,
12     TRIM(cst_firstname) AS cst_firstname,
13     TRIM(cst_lastname) AS cst_lastname,
14     CASE WHEN UPPER(TRIM(cst_material_status)) = 'S' THEN 'Single'
15           WHEN UPPER(TRIM(cst_material_status)) = 'M' THEN 'Married'
16           ELSE 'n/a'
17     END cst_material_status,
18     CASE WHEN UPPER(TRIM(cst_gndr)) = 'F' THEN 'FEMALE'
19           WHEN UPPER(TRIM(cst_gndr)) = 'M' THEN 'MALE'
20           ELSE 'n/a'
21     END cst_gndr,
22     cst_create_date
23 FROM (
24     SELECT *,
25         ROW_NUMBER() OVER (PARTITION BY cst_id ORDER BY cst_create_date
26                             DESC) AS flag_last
27     FROM bronze.crm_cust_info
28     WHERE cst_id IS NOT NULL
29 ) t
```

```
29 WHERE flag_last = 1
```

### Table: crm\_prd\_info

#### 1. Checking for Nulls or Duplicates in the Primary Key (prd\_id)

```
1 SELECT prd_id, COUNT(*)
2 FROM silver.crm_prd_info
3 GROUP BY prd_id
4 HAVING COUNT(*) > 1 OR prd_id IS NULL
```

**2. Splitting the prd\_key Column** The prd\_key column contains combined data: the **Category ID** (first 5 characters) and the **Product ID** (remaining characters). This column is split into two separate columns, cat\_id and prd\_key, so that the table can later be connected to the sales table using prd\_key:

```
1 SELECT
2     prd_id,
3     prd_key,
4     REPLACE(SUBSTRING(prd_key, 1, 5), '-', '_') AS cat_id,
5     SUBSTRING(prd_key, LEN(prd_key)) AS prd_key,
6     prd_nm,
7     prd_cost,
8     prd_line,
9     prd_start_dt,
10    prd_end_dt
11 FROM bronze.crm_prd_info
```

#### 3. Checking for Unwanted Spaces (prd\_nm)

```
1 SELECT prd_nm
2 FROM bronze.crm_prd_info
3 WHERE prd_nm != TRIM(prd_nm)
```

#### 4. Checking for Nulls or Negative Numbers (prd\_cost)

```
1 SELECT prd_cost
2 FROM silver.crm_prd_info
3 WHERE prd_cost < 0 OR prd_cost IS NULL
```

NULL values are replaced by 0.

**5. Data Standardization and Consistency (prd\_line)** Brief/abbreviated values in the prd\_line column are replaced with user-friendly values, following the same logic applied to the gender and marital status columns.

**6. Checking for Invalid Date Orders (prd\_start\_dt and prd\_end\_dt)** When facing complex data issues, the recommended workflow is:

1. Extract a few examples into Excel.
2. Identify the logic manually.
3. Validate the logic with business experts.
4. Convert the validated logic to SQL.

Excel is preferred at this stage because it is easier to explain and discuss than database queries.

**The problem:** in some records, the start date is *after* the end date, which is logically impossible:

| Product   | prd_start_dt | prd_end_dt | Problem     |
|-----------|--------------|------------|-------------|
| Product A | 2018-01-01   | 2011-01-01 | Start > End |
| Product B | 2012-01-01   | 2007-01-01 | Start > End |

Table 5.1: Invalid date orders in crm\_prd\_info

**Attempt 1 – Just Swap the Dates (Failed):** simply swapping the start and end dates creates overlapping date ranges between consecutive records for the same product:

| Product              | New Start  | New End    | Issue                              |
|----------------------|------------|------------|------------------------------------|
| Product A            | 2011-01-01 | 2018-01-01 | Works                              |
| Product A (next row) | 2012-01-01 | 2018-01-01 | Overlap! Both rows cover 2012–2018 |

Table 5.2: Failed attempt: swapping start and end dates

This violates the rule that the end of one record must be *before* the start of the next record (no overlapping periods).

**The Real Solution – Rebuild the End Date from the Next Start Date:** the end date of the current record is derived from the start date of the *next* record, with a one-day adjustment to avoid overlap:

END\_DATE = (NEXT start date) - 1 day

| Record   | Start Date | End Date (adjusted)                |
|----------|------------|------------------------------------|
| Record 1 | 2007-01-01 | 2011-12-31 (day before next start) |
| Record 2 | 2012-01-01 | 2017-12-31 (day before next start) |
| Record 3 | 2018-01-01 | NULL (current – no end date yet)   |

Table 5.3: Corrected date ranges using the LEAD-based approach

**The SQL Solution – the LEAD() Window Function:** LEAD() accesses data from the *next* row within the same partition (as opposed to LAG(), which looks at the *previous* row):

```
1 LEAD(column_name) OVER (PARTITION BY group_column ORDER BY sort_column)
```

Table: crm\_sales\_details

### 1. Checking for Invalid Dates

```
1 -- Check for Invalid Dates
2 SELECT
3     NULLIF(sls_order_dt, 0) sls_order_dt
4 FROM bronze.crm_sales_details
5 WHERE sls_order_dt <= 0
6     OR LEN(sls_order_dt) != 8
```

### 2. Checking for Invalid Date Orders

```
1 -- Check for Invalid Date Orders
2 SELECT *
3 FROM bronze.crm_sales_details
4 WHERE sls_order_dt > sls_ship_dt
5     OR sls_order_dt > sls_cust_id
```

**3. Checking Data Consistency Between Sales, Quantity and Price** The business rule is that  $Sales = Quantity \times Price$ , and none of these three values should be NULL, zero, or negative:

```

1 -- Check Data Consistency: Between Sales, Quantity and Price
2 -- Sales = Quantity * Price
3 -- Values must not be NULL, zero, or Negative
4 SELECT
5     sls_sales,
6     sls_quantity,
7     sls_price
8 FROM bronze.crm_sales_details
9 WHERE sls_sales != sls_quantity * sls_price
10      OR sls_sales IS NULL OR sls_quantity IS NULL OR sls_price IS NULL
11      OR sls_sales <= 0 OR sls_quantity <= 0 OR sls_price <= 0

```

**General principle:** when data issues are discovered, business experts must be consulted first. They will either fix the issue at the source, or provide the rules according to which the data should be cleaned – cleansing decisions should never be based on the data engineer’s own assumptions.

#### Table: erp\_cust\_az12

**1. Column cid** Based on data analysis and integration work, the table `erp_cust_az12` is connected to `crm_cust_info` via `cid ↔ cst_key`. However, the values are not exactly identical: `cid` contains an extra prefix `NAS` at the beginning. This prefix is removed so that the two tables can be joined correctly:

```

1 SELECT
2     CASE WHEN cid LIKE 'NAS%' THEN SUBSTRING(cid, 4, LEN(cid))
3     ELSE cid
4     END cid,
5     bdate,
6     gen
7 FROM bronze.erp_cust_az12

```

**2. Column bdate** It is verified that the column type is `DATE`, and that there are no unrealistically old dates or future dates. Any invalid dates are replaced with `NULL`, or discussed with the source system manager to determine how to handle them.

**3. Column gen** The distinct values of the `gen` column are inspected:

```

1 SELECT DISTINCT gen
2 FROM bronze.erp_cust_az12

```

The values are then standardized into only three categories: `Male`, `Female`, and `n/a`:

```

1 CASE WHEN UPPER(TRIM(gen)) IN ('F', 'FEMALE') THEN 'Female'
2      WHEN UPPER(TRIM(gen)) IN ('M', 'MALE') THEN 'Male'
3      ELSE 'n/a'
4 END AS gen

```

#### Table: erp\_loc\_a101

**1. Column cid** Based on the data analysis, it was assumed that `crm_cust_info` and `erp_loc_a101` are connected via `cst_key ↔ cid`. After verifying this assumption, it was found that the `cid` values in `erp_loc_a101` contain an extra `'-'` character after `'AW'`, which is not present in `crm_cust_info`. This character is removed:

```
1 REPLACE(cid, '-', '') AS cid
```

**2. Column cntry** All distinct values of the `cntry` column are inspected and then standardized using a CASE WHEN expression:

```
1 CASE WHEN TRIM(cntry) = 'DE' THEN 'Germany'
2       WHEN TRIM(cntry) IN ('US', 'USA') THEN 'United States'
3       WHEN TRIM(cntry) = '' OR cntry IS NULL THEN 'n/a'
4       ELSE TRIM(cntry)
5 END
```

**3. Loading into Silver** The cleaned data is inserted into the corresponding Silver-layer table.

**4. Final Quality Check in the Silver Layer** A final quality check is performed on the Silver-layer table to ensure all the previous rules have been applied correctly.

**Table: erp\_px\_cat\_g1v2**

**1. Column id** The tables `erp_px_cat_g1v2` and `crm_prd_info` are connected via `id` ↔ `cat_id`. After verification, the values match correctly with no issues.

**2. Columns cat\_id, subcat, maintenance** These columns are checked for unwanted spaces and for non-standardized (brief) values. No issues were found.

**3. Loading into Silver** The verified data is inserted directly into the Silver layer.

#### 5.3.4 Assembling the Silver Layer Procedure

Finally, all the individual cleansing scripts described above are assembled into a single stored procedure, `load_silver`, which performs the full load of the Silver layer in one execution. The overall data flow diagram is also extended to reflect the Bronze-to-Silver transformations.

## 5.4 Building the Gold Layer

The **Gold layer** is the business-focused layer of the data warehouse.

### 5.4.1 Bronze/Silver vs. Gold

| Layer(s)        | Focus                                                                                                                      |
|-----------------|----------------------------------------------------------------------------------------------------------------------------|
| Bronze & Silver | Technical only – data ingestion, cleansing, quality control. Tables remain source-system oriented.                         |
| Gold            | Business-focused – breaks away from the source data models and builds something completely new for reporting and analysis. |

Table 5.4: Difference between Bronze/Silver and Gold layers

Before building the Gold layer, it is essential to understand the business and its processes: the Gold layer redesigns the data *for the business*, not for the source systems.

### 5.4.2 Data Modeling

Data modeling takes the messy, unstructured raw data and organizes it into clean, business-friendly objects (Customers, Orders, Products) with clear relationships between them.

| Model Type | Focus             | Content                                                                           |
|------------|-------------------|-----------------------------------------------------------------------------------|
| Conceptual | Big picture only  | Entities (Customers, Orders, Products) and their relationships.                   |
| Logical    | What data we need | Entities + columns (e.g., Customer ID, Name) + keys + relationships.              |
| Physical   | How to implement  | Everything from the logical model, plus data types, lengths, and storage details. |

Table 5.5: Levels of data modeling

**Practical approach:** in this project, only conceptual and logical models were drawn. Physical models require significantly more effort and are typically auto-generated by tools such as Databricks.

### 5.4.3 Documentation Types

- **Data Model Diagram:** a visual representation of entities and their relationships.
- **Data Dictionary:** describes the data model in detail.
- **Data Flow Diagram:** shows how data moves through the system.

### 5.4.4 Facts and Dimensions

- **Fact table:** represents an *event* (e.g., a sale happened). It stores measurements – numeric values such as sales, quantity, and revenue.
- **Dimension table:** represents the *context* of an event (who bought, what, where, when). It stores attributes and descriptive text, such as names, addresses, and categories.

### 5.4.5 Creating the Customer Dimension (dim\_customers)

The customer dimension is built from the Silver-layer tables. The master table is `crm_cust_info`, which is then left-joined with `erp_cust_az12` and `erp_loc_a101`. A **left join** is used rather than an inner join, because not all customers are present in all three tables.

**1. Checking for Duplicates** A duplicate check on the joined result confirmed no issues.

**2. Resolving Contradictory Data (Gender)** Two columns indicate the customer's gender: `cst_gndr` (from CRM) and `gen` (from ERP). The base query joining the three tables is:

```

1 SELECT
2     ci.cst_id,
3     ci.cst_key,
4     ci.cst_firstname,
5     ci.cst_lastname,
6     ci.cst_material_status,
7     ci.cst_gndr,
8     ci.cst_create_date,
9     ca.bdate,
10    ca.gen,

```



```
11     la.cntry
12 FROM silver.crm_cust_info ci
13 LEFT JOIN silver.erp_cust_az12 ca
14     ON ci.cst_key = ca.cid
15 LEFT JOIN silver.erp_loc_a101 la
16     ON ci.cst_key = la.cid
```

The two gender columns are first isolated and compared. The resolution logic is as follows:

- If the master table (CRM) contains a value, that value is used.
- Otherwise, if the ERP table contains a value, that value is used.
- Otherwise, the value is set to 'n/a'.

```
1 SELECT
2     ci.cst_gndr,
3     ca.gen,
4     CASE WHEN ci.cst_gndr != 'n/a' THEN ci.cst_gndr -- CRM is the Master
5           for gender info
6           ELSE COALESCE(ca.gen, 'n/a')
7 END AS new_gen
```

COALESCE returns the first non-NULL value from a list of arguments.

**3. Renaming and Reordering Columns** Columns are renamed to friendly, business-aligned names and reordered into a more logical sequence.

**4. Classifying the Table** This table contains descriptive information about the customer (the object), without any events or measures. It is therefore classified as a **customer dimension table**.

**5. Creating the Surrogate Key** A **surrogate key** is an artificially created unique identifier for a row in a table – it has no business meaning and never changes. Since the source tables do not provide a reliable primary key, a surrogate key is generated using the ROW\_NUMBER() window function (this can also be done at the DDL level):

```
1 SELECT
2     ROW_NUMBER() OVER (ORDER BY cst_id) AS customer_key,
3     ci.cst_id AS customer_id,
4     ci.cst_key AS customer_number,
5     ...
```

**6. Creating the View** A **view** is a virtual table that stores a saved SQL query:

```
1 CREATE VIEW gold.dim_customers AS
2 SELECT
3     ...
```

*Note:* a view is a read-only virtual table based on a saved SELECT query, whereas a stored procedure is a saved script that can accept parameters, contain complex logic (conditions, loops, error handling), and modify data. Views are used for reporting, while stored procedures are used for ETL loading.

### 5.4.6 Creating the Product Dimension (dim\_products)

The product dimension starts from the CRM product table and is left-joined with the corresponding ERP category table.

- Historical records are filtered out by keeping only rows where `prd_end_dt` IS NULL.
- Uniqueness is verified, in case the ERP table introduces duplicates:

```

1 SELECT prd_key, COUNT(*) FROM (
2     SELECT
3         pn.prd_id,
4         pn.cat_id,
5         pn.prd_key,
6         pn.prd_nm,
7         pn.prd_cost,
8         pn.prd_line,
9         pn.prd_start_dt,
10        pc.cat,
11        pc.subcat,
12        pc.maintenance
13    FROM silver.crm_prd_info pn
14    LEFT JOIN silver.erp_px_cat_glv2 pc
15        ON pn.cat_id = pc.id
16    WHERE prd_end_dt IS NULL -- Filter out all historical data
17 ) t
18 GROUP BY prd_key
19 HAVING COUNT(*) > 1

```

After checking for column redundancy, reordering, and renaming the columns, and creating a surrogate key, the final view is created:

```

1 CREATE VIEW gold.dim_products AS
2 SELECT
3     ROW_NUMBER() OVER (ORDER BY pn.prd_start_dt, pn.prd_key) AS
4         product_key,
5     pn.prd_id AS product_id,
6     pn.prd_key AS product_number,
7     pn.prd_nm AS product_name,
8     pn.cat_id AS category_id,
9     pc.cat AS category,
10    pc.subcat AS subcategory,
11    pc.maintenance,
12    pn.prd_cost AS cost,
13    pn.prd_line AS product_line,
14    pn.prd_start_dt AS start_date
15 FROM silver.crm_prd_info pn
16 LEFT JOIN silver.erp_px_cat_glv2 pc
17     ON pn.cat_id = pc.id
18 WHERE prd_end_dt IS NULL -- Filter out all historical data

```

### 5.4.7 Creating the Sales Fact Table (fact\_sales)

The sales table is classified as a **fact table** because:

- It contains multiple foreign keys connecting it to several dimensions.
- It contains many date-related fields, representing events.

- It contains numerical measures and metrics (sales amount, quantity, price).

A key principle when building the fact table is that the **surrogate keys** from the dimension tables must be used to connect the fact table to its dimensions, instead of the original source-system identifiers.

```

1 SELECT
2     sd.sls_ord_num ,
3     pr.product_key ,
4     cu.customer_key ,
5     sd.sls_order_dt ,
6     sd.sls_ship_dt ,
7     sd.sls_due_dt ,
8     sd.sls_sales ,
9     sd.sls_quantity ,
10    sd.sls_price
11 FROM silver.crm_sales_details sd
12 LEFT JOIN gold.dim_products pr
13     ON sd.sls_prd_key = pr.product_number
14 LEFT JOIN gold.dim_customers cu
15     ON sd.sls_cust_id = cu.customer_id

```

The remaining steps for the fact table are:

1. Rename and reorder the columns.
2. Create the view `gold.fact_sales`.
3. Perform a final quality check.

#### 5.4.8 Building the Star Schema Model

Once `dim_customers`, `dim_products`, and `fact_sales` are defined, they together form a **star schema**: the fact table at the center, connected to its surrounding dimension tables through surrogate keys.

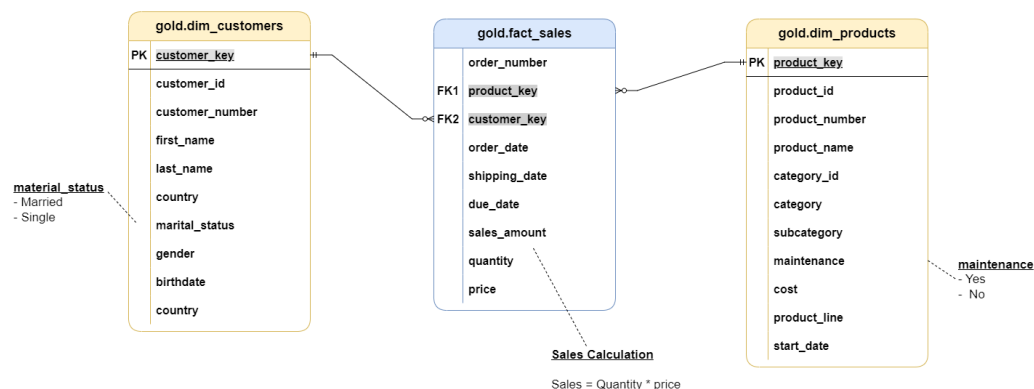


Figure 5.2: Star Schema Model

#### 5.4.9 Creating the Data Catalog

A **data catalog** is a document describing, for each table in the Gold layer:

- The purpose and content of the table.
- The columns, with descriptions and data types.

- The relationships between tables.
- Examples of actual values (e.g., `male/female/not available` instead of simply “gender”), which are crucial for understanding the data.

The data catalog is then shared with data consumers (analysts, business users) to facilitate the correct use of the Gold layer.

#### 5.4.10 Finalizing the Data Flow Diagram

As a final step, the overall **data flow diagram (lineage)** is completed, documenting the full journey of data from the CRM and ERP source systems, through the Bronze and Silver layers, into the final Gold-layer star schema.

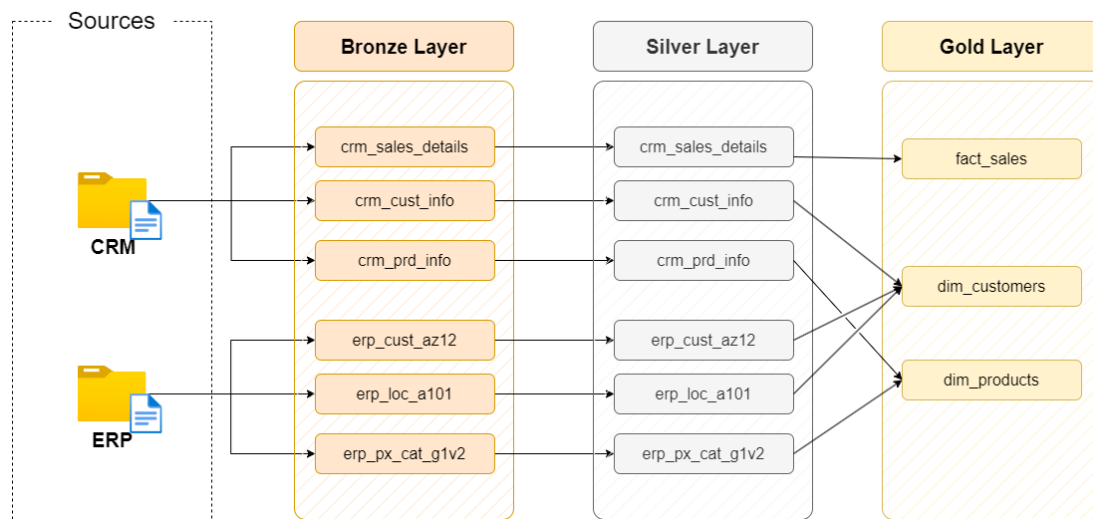


Figure 5.3: Data Flow Diagram

## Chapter 6

# Conclusion

This project consisted of designing and implementing a data warehouse from scratch, integrating two source systems (CRM and ERP), each composed of three tables, using the **Medallion Architecture** (Bronze, Silver, Gold).

The Bronze layer provided a faithful, unmodified copy of the source data, ensuring full traceability. The Silver layer applied systematic data cleansing, standardization, and integration rules – including deduplication with window functions, whitespace trimming, value standardization, date-range correction using `LEAD()`, and key harmonization between CRM and ERP identifiers. Finally, the Gold layer redefined the data from a business perspective, producing a clean **star schema** composed of `dim_customers`, `dim_products`, and `fact_sales`, accompanied by a data catalog and a complete data flow diagram.

Throughout the project, strict **naming conventions** and the principle of **Separation of Concerns** ensured consistency, maintainability, and traceability across all layers. The resulting architecture provides a single, reliable, and well-documented source of truth, ready to support reporting, analytics, and future machine learning use cases.